UCL Robot Perception and Learning Lab

Department of Computer Science

Innovation Lab—Robotic Manipulation

Michael Ziegltrum, Dr. Valerio Modugno, Professor Dimitrios Kanoulas

Goals

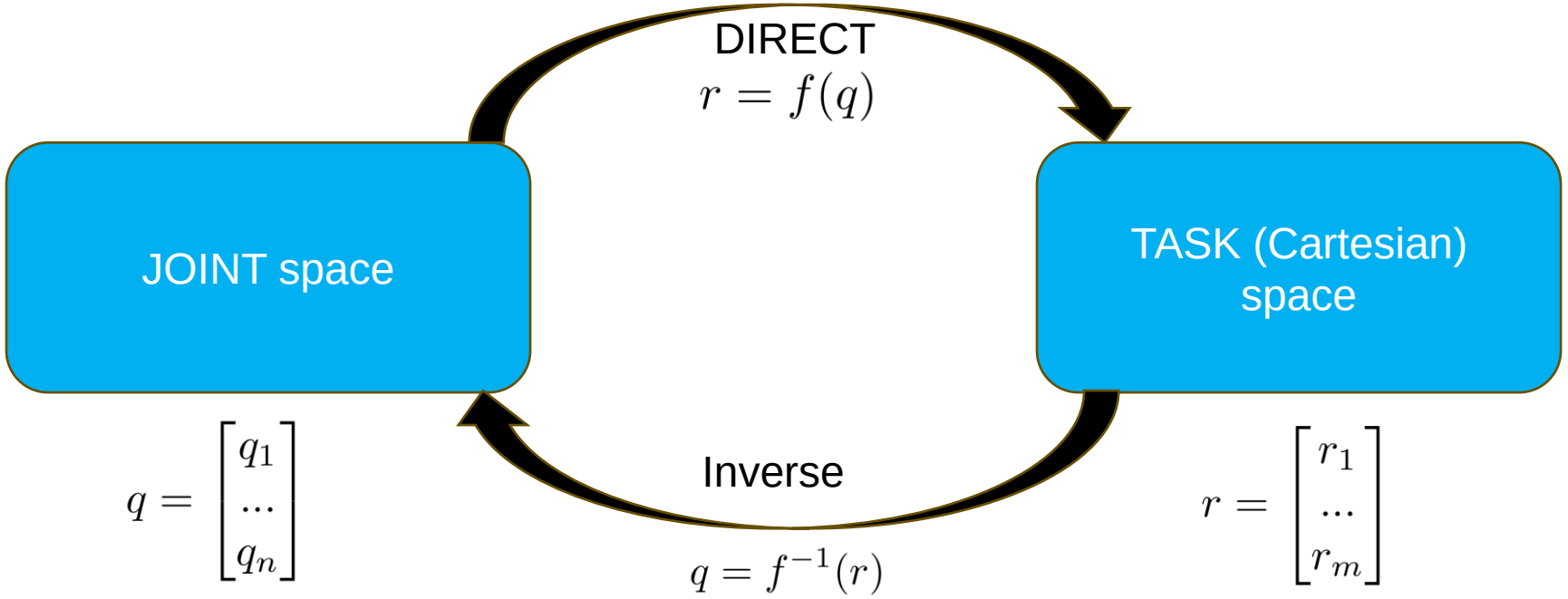
- Recall Vocabulary and Knowledge (Robotic Manipulators)
- Learn to use the myCobot 280 Robotic Arm
- Write scripts to accomplish manipulation tasks

Goals

- **Recall Vocabulary and Knowledge (Robotic Manipulators)**
- Learn to use the myCobot 280 Robotic Arm
- Write scripts to accomplish manipulation tasks

Recall: Kinematics

- We choose how to represent q (parametrization)
  - Ideally unambigious and minimal
  - n = # degrees of freedom (dof) = # robot joints (rotational or translational)
- Choice of parametrization r
  - Compact, describes position and/or orientation (pose) variables of interest to the required task
  - Usually m <= n, m=6

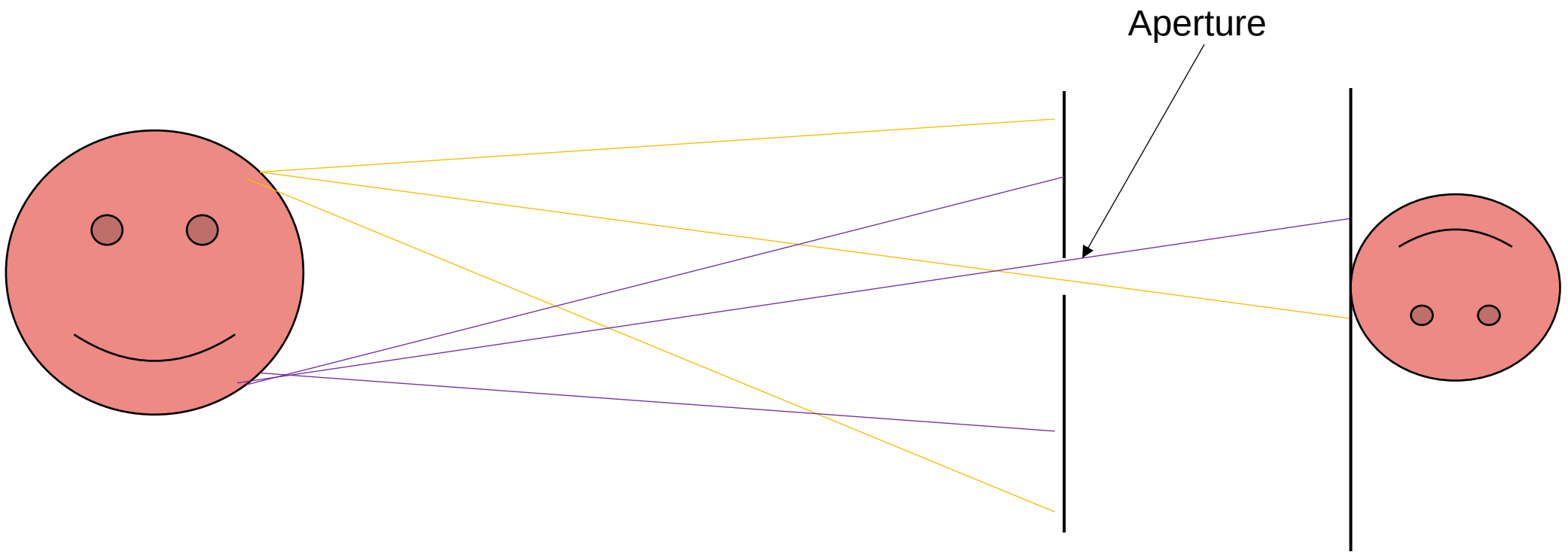


Recall: Differential Kinematics

- Relations between motion (velocity) in joint space and motion (linear/angular velocity) in task space (e.g., cartesian space).
- Instantaneous velocity mappings can be obtained through time differentiation of the direct kinematics or in a geometric way, directly at the differential level

Recall: Pinhole Camera Model

- Add a barrier to block off most of the rays
- Pinhole camera, without a lens but with a tiny aperture—a pinhole



Recall: Pinhole Camera Model

- Camera produces a matrix of RGB color vectors

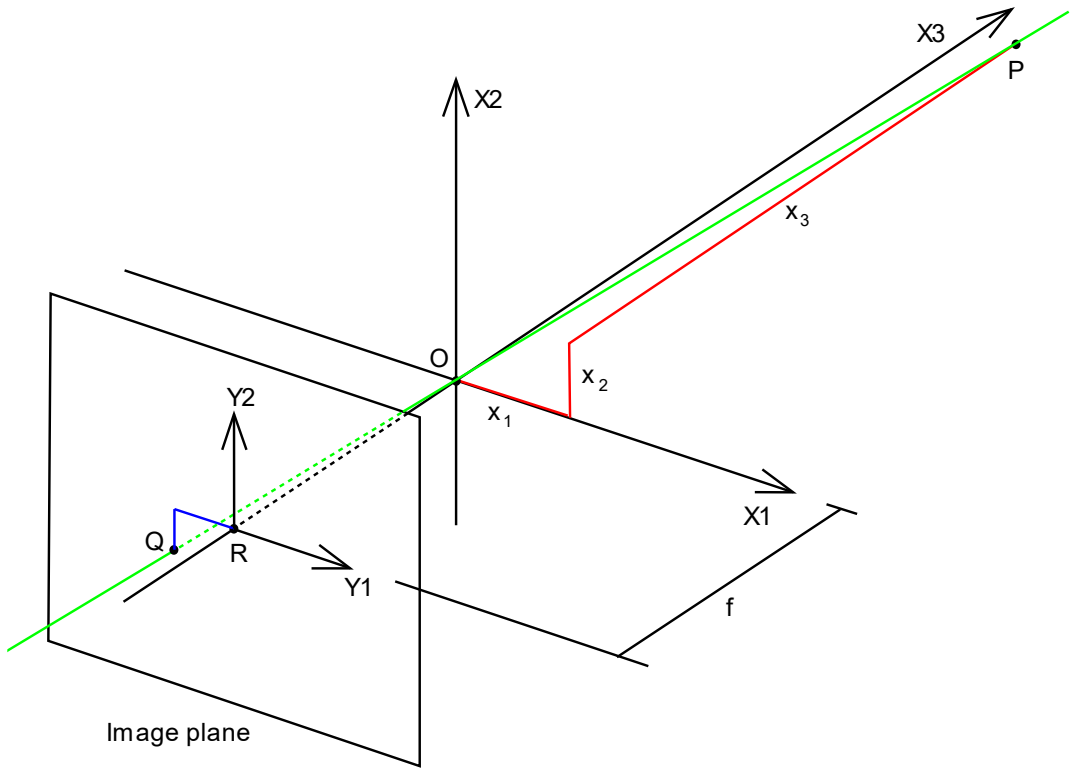
$$\begin{bmatrix} c_{1,1} & \cdots & c_{1,N} \\ \vdots & \ddots & \vdots \\ c_{M,1} & \cdots & c_{M,N} \end{bmatrix}, c_{u,v} = (r_{u,v} g_{u,v} b_{u,v})$$

- Need to map from camera frame at (p_x, p_y, p_z) to the pixel coordinates (u, v) that have color $c_{u,v}$
- Use the pinhole camera model and a depth camera
- Depends on parameters intrinsic to the camera, like distance from image plane to aperature (focal length)
- $p^h = [K \quad 0_{3x1}] P_C^h$

Point p in homogenous pixel coordinates

Camera intrinsics matrix

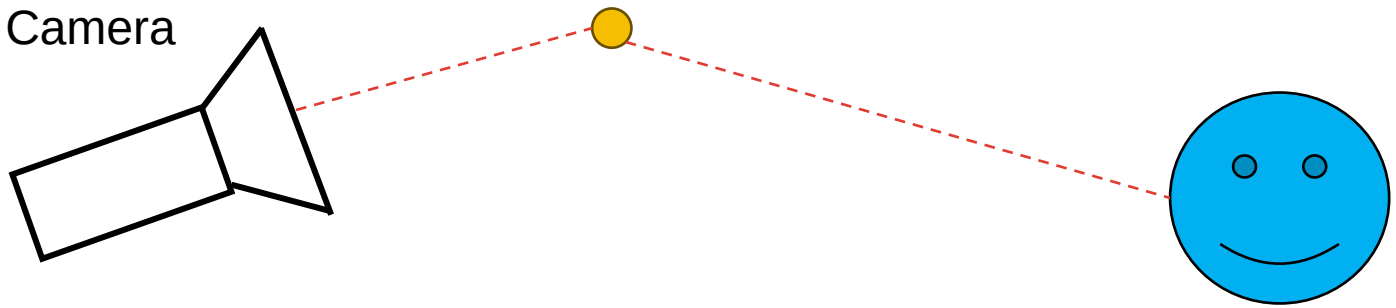
Point P_C in homogenous camera coordinates



Images from Wikipedia Commons

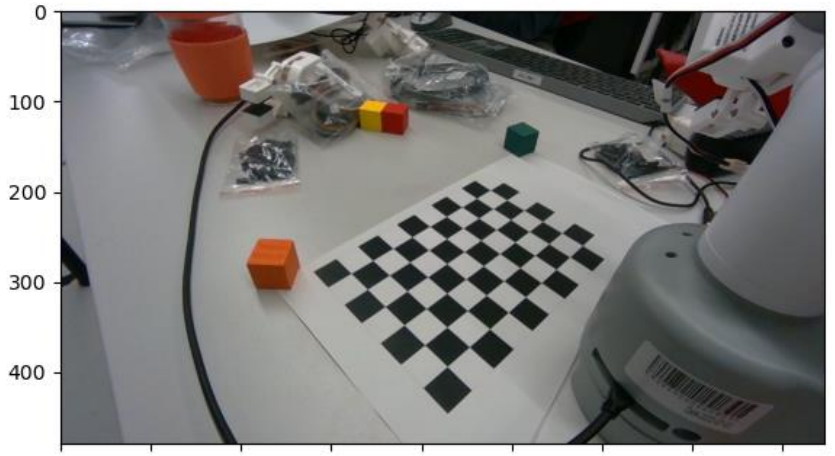
Recall: World Coordinates

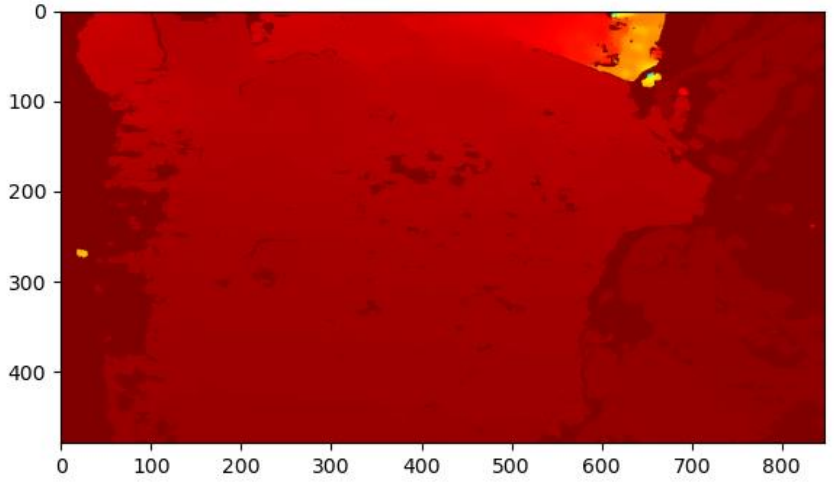
- Lastly, we need to go from camera coordinates to world coordinates
- We do this often with a rotation matrix R, and a translation matrix t
- $P_C = t + R\ P_W$



Recall: Computer Vision—In Practice

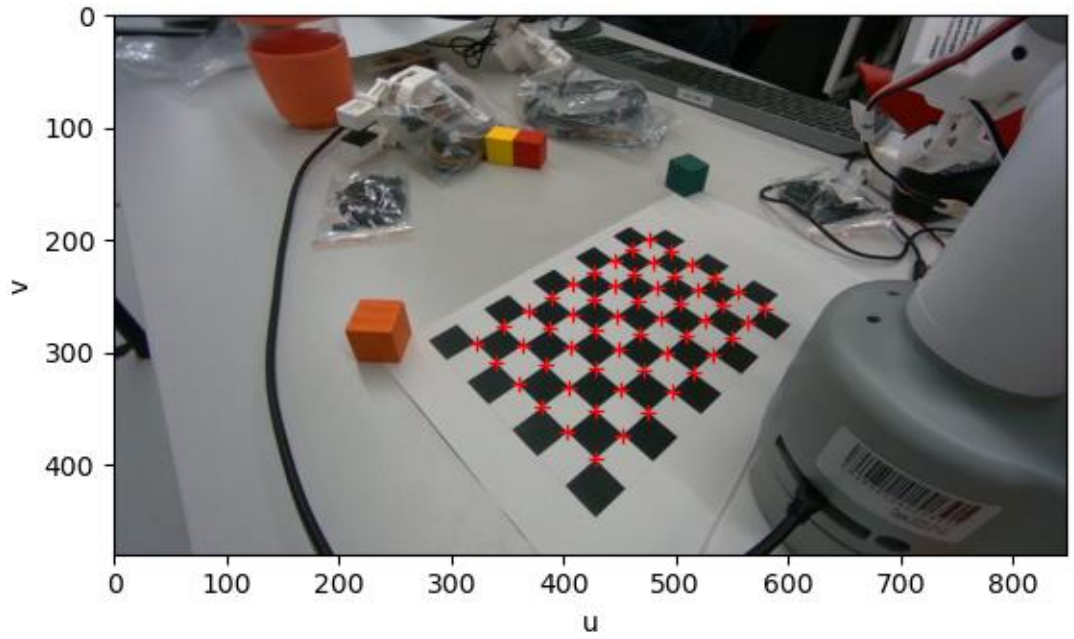
- We get a pair of color and depth images from the camera
- We find the object we are interested in in pixel coordinates
- We get the coordinates of the object in camera frame coordinates





Recall: Computer Vision—In Practice

- We transform the camera frame coordinates to world frame coordinates using prior calibration (extrinsics)

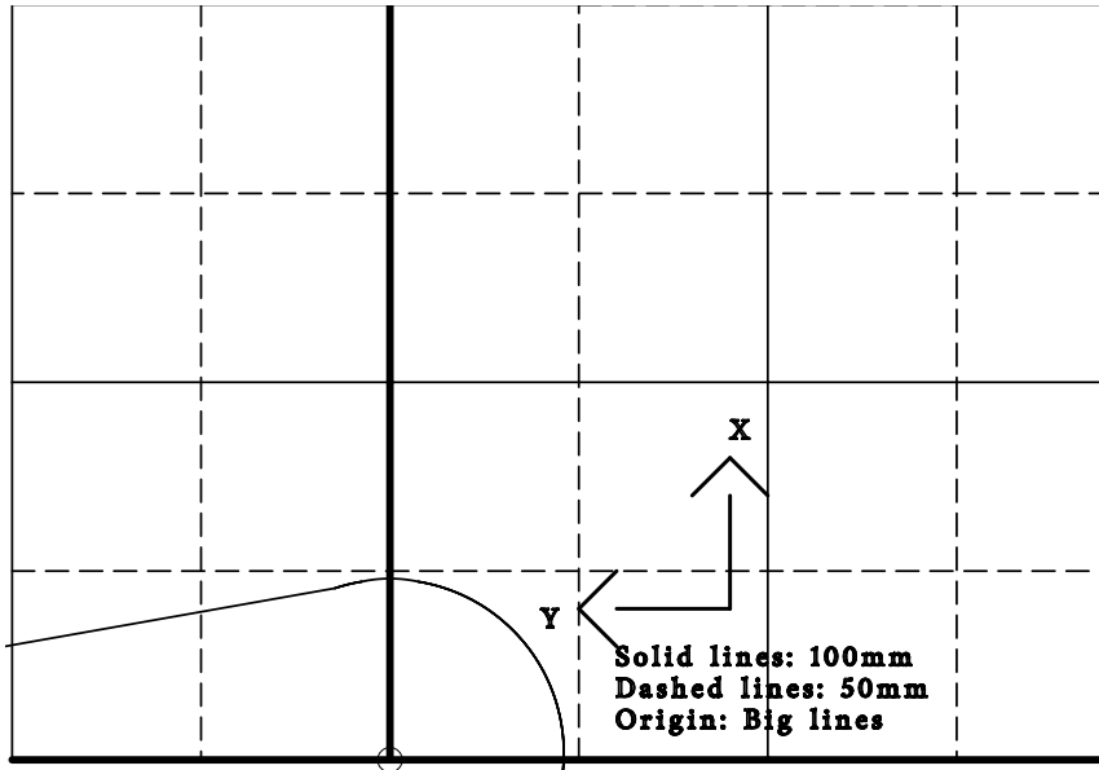


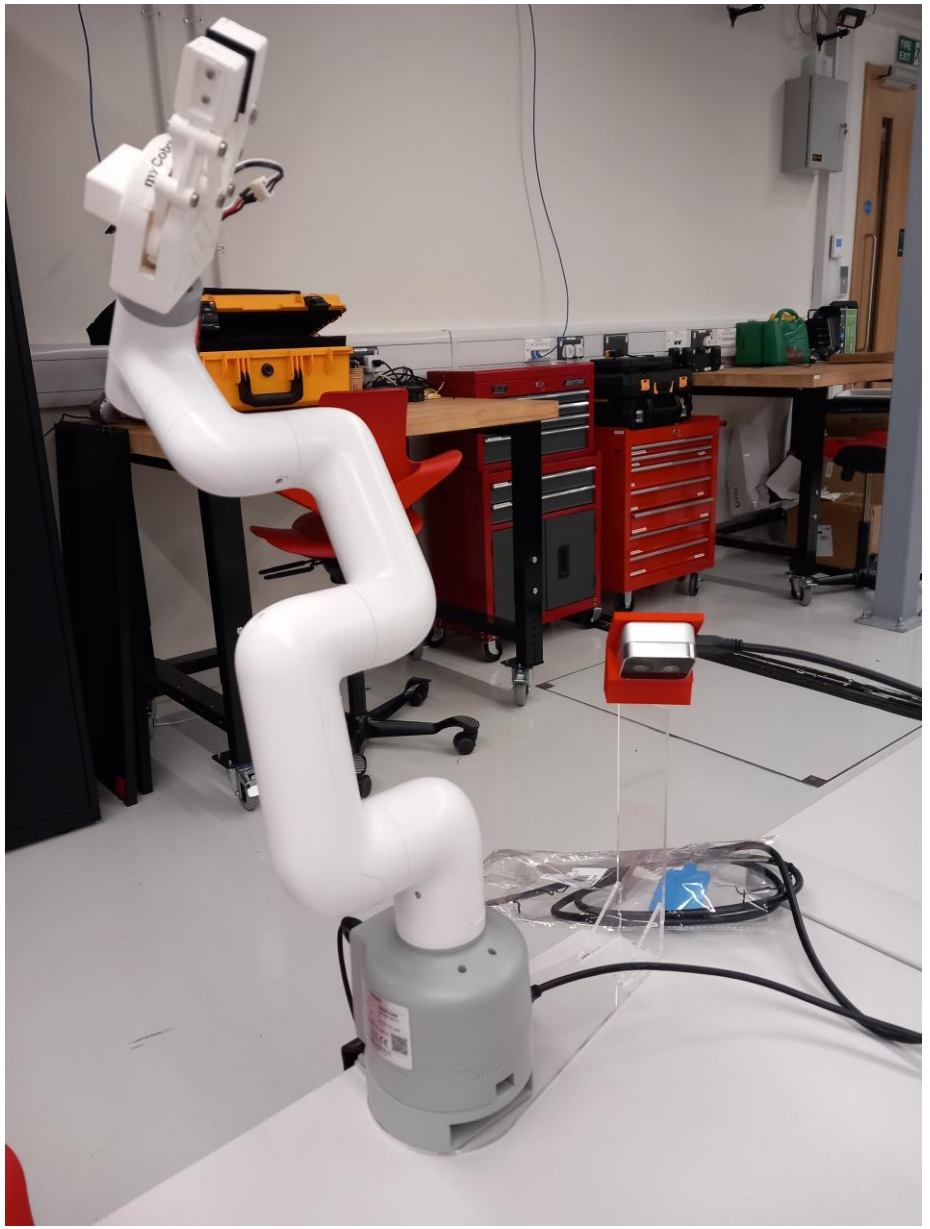
Goals

- Recall Vocabulary and Knowledge (Robotic Manipulators)
- **Learn to use the myCobot 280 Robotic Arm**
- Write scripts to accomplish manipulation tasks

The Setup

- Robot Arm
- Depth Camera
- Gripper
- Grid





myCobot—Elephant Robotics

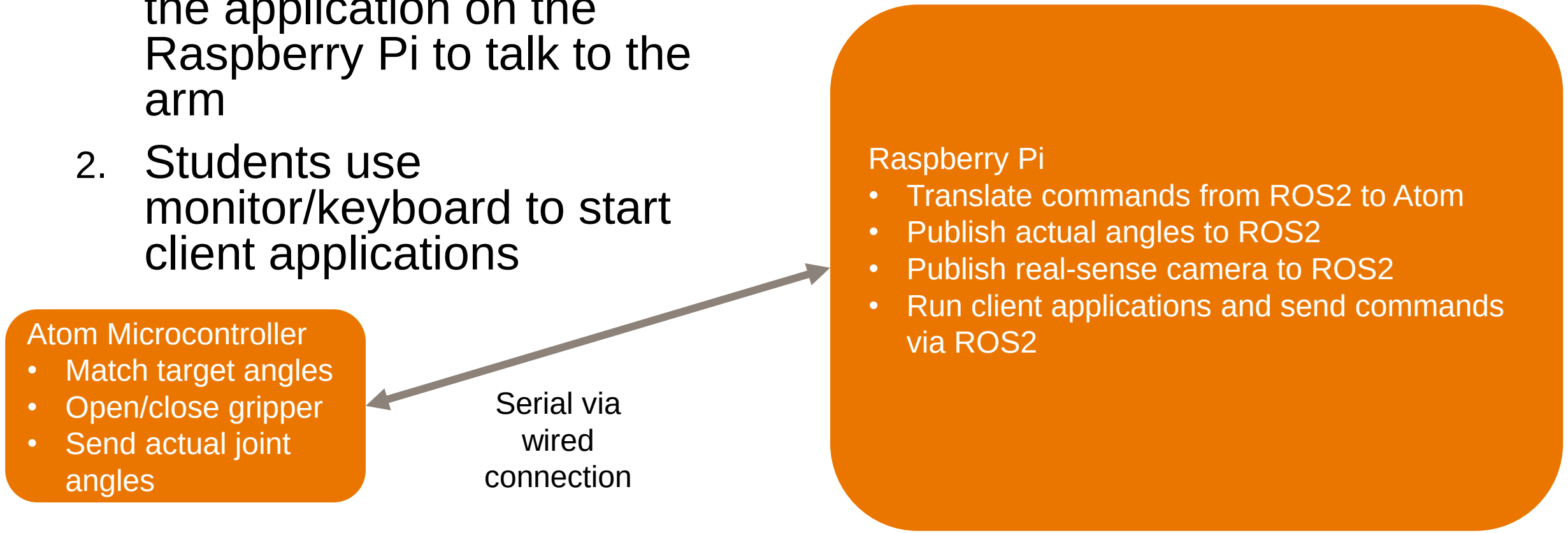
- UCL has invested in educational robots including the myCobot 280 Raspberry Pi Edition from Elephant Robotics
- 6 DOF robot arm with servo motors
- Runs Ubuntu Desktop on the built-in Raspberry Pi
- Raspberry Pi communicates with microcontroller (Atom) that does low-level control. It uses a special protocol for this
- Effective radius ~280mm—more arm parameters here

myCobot—Safety

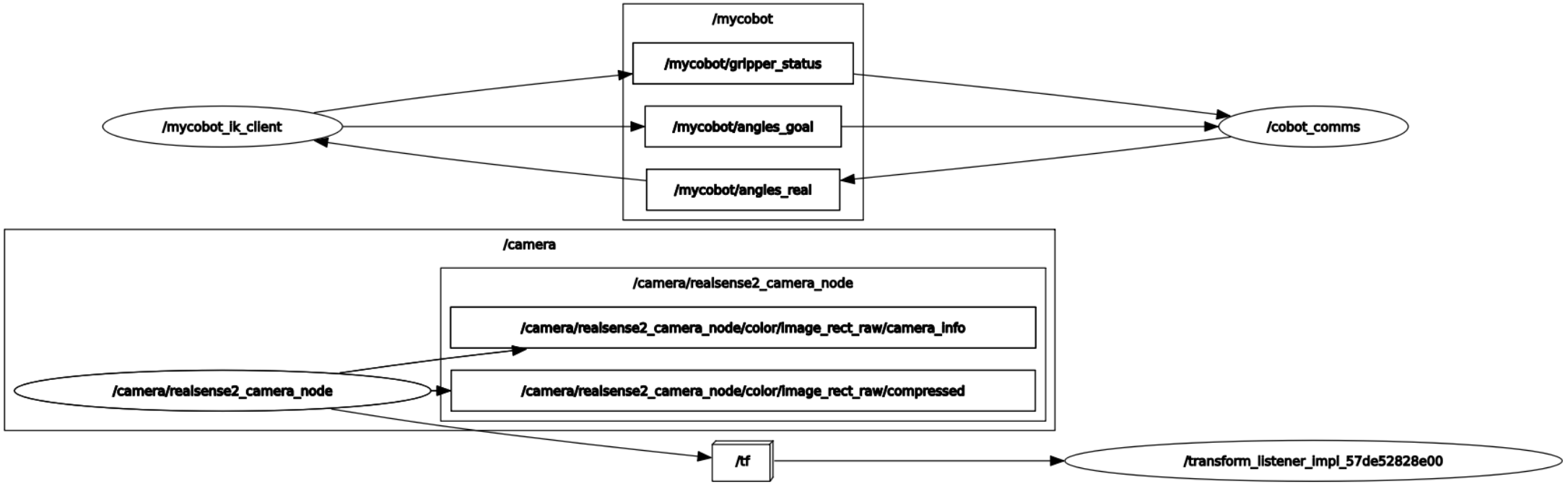
- Robots can behave unexpectedly (to humans)
- During each task dedicate 1 group member to monitor
  - Is someone too close to the robot?
  - Are fingers in between joints?
  - Is the robot going to crash into something?
  - Is the robot going to crash into itself?
  - Is it making strange noises?
- Be aggressive with the e-stop and call over a supervisor

myCobot—How to Control the Arm?

1. Students use monitor/keyboard to start the application on the Raspberry Pi to talk to the arm
2. Students use monitor/keyboard to start client applications



myCobot—Our Setup



myCobot—Starting the Comms Node

- We need to connect to the robot (keyboard/mouse/monitor or ssh)
- Ssh: if linux/mac, built in ssh support. Else, download a tool like MobaXterm
- Connect to robot WiFi—“mycobot_23”, password: autonomy
- Robot ip address: 10.42.0.1, user: er, password: elephant (or Elephant) —“ssh er@10.42.0.1”

myCobot—Starting the Comms Node

- Prior students may have done work on the robot
- Clear it out using git:

```
cd /home/er/ros_ws/src/mycobot_client
git restore *
```

myCobot—Starting the Comms Node

- We use docker to run the application on the Pi
  - This is like using a virtual machine in a “loosely isolated environment called a container”
  - This makes it easy for us to setup a new arm
  - It also makes it more independent to OS configuration
- README here, but abbreviated commands below (third command is one line, enter what is inside the quotes)

```
mycobot-up
export ROS_DOMAIN_ID=10
“ros2 launch mycobot_interface_2 mycobot_comms_launch.py
use_realsense:=False”
```

Bash alias for our (long) docker run command

ROS2 uses this to identify which group of computers should be networked

ROS2 launch file to bring up our application to talk to the arm

myCobot—Starting the Client Node

- README here, ROS tutorials here are useful
- Control the arm via a script
- `mycobot-client-up`
- `export ROS_DOMAIN_ID=10`
- `colcon build`
- `source install/setup.bash`
- `ros2 run mycobot_client_2 cobot_ik_demo`

myCobot—Starting the App on the Client (Continued)

- Control the arm via your script
- Open a file explorer. Go to ~/ros_ws/src/mycobot_client/mycobot_client_2/mycobot_client_2/ and open the run_task.py file with the Ubuntu text editor.
- `colcon build`
- `export ROS_DOMAIN_ID=10`
- `source install/setup.bash`
- `ros2 run mycobot_client_2 run_task`

myCobot—CobotIK Class

Classes

rclpy.node.Node(builtins.object)
    CobotIK

class **CobotIK**(rclpy.node.Node)

```
CobotIK(speed: int = 30)

Class that exposes functions to talk to the robot arm given numpy arrays. It translates these to the needed ros messages. It also calculates IK.
```

Method resolution order:
- CobotIK
- rclpy.node.Node
- builtins.object

Methods defined here:

__init__(self, speed: int = 30)

```
Initialize the sim and such.

Args:
    speed (int, optional): how fast should the arm move when given commands. 0-100. Defaults to 30.

Raises:
    ValueError: if speed is wrong.
```

myCobot—CobotIK Class

calculate_ik(self, target_position: numpy.ndarray[typing.Any, numpy.dtype[float]], euler_angles_degrees: Optional[numpy.ndarray[Any, numpy.dtype[float]]] = None, target_frame: Optional[str] = None) -> numpy.ndarray[typing.Any, numpy.dtype[float]]

```
calculate what angles the robot joints should have to reach the targets. Takes angles in degrees.

Args:
    target_position (npt.NDArray[float]): x, y, z
    euler_angles_degrees (Optional[npt.NDArray[float]], optional): rx, ry, rz, or None. Easier to solve if None. Defaults to None.
    target_frame (Optional[str], optional): what frame to use for the calcs. Defaults to None.

Returns:
    npt.NDArray[float]: the joint angles
```

close_gripper(self)

```
Close the gripper.
```

get_pose(self, cur_joint_angles: Optional[numpy.ndarray[Any, numpy.dtype[float]]] = None, target_frame: str = 'gripper') -> Tuple[numpy.ndarray[Any, numpy.dtype[float]], numpy.ndarray[Any, numpy.dtype[float]]]

```
Helper function to get the current pose of the robot from the current angles.

Args:
    cur_joint_angles (Optional[npt.NDArray[float]], optional): What current angles to calculate direct kinematics with? 6D numpy array. Defaults to None.. Defaults to None.
    target_frame (str, optional): What frame should the pose be in, from frames in the URDF. Defaults to "gripper".. Defaults to "gripper".

Returns:
    Tuple[npt.NDArray[float], npt.NDArray[float]]: _description_
```

get_real_angles(self) -> numpy.ndarray[typing.Any, numpy.dtype[float]]

```
Helper function to return the current angles.

Returns:
    npt.NDArray[float]: numpy array with the joint angles 1-6.
```

open_gripper(self)

```
Open the gripper.
```

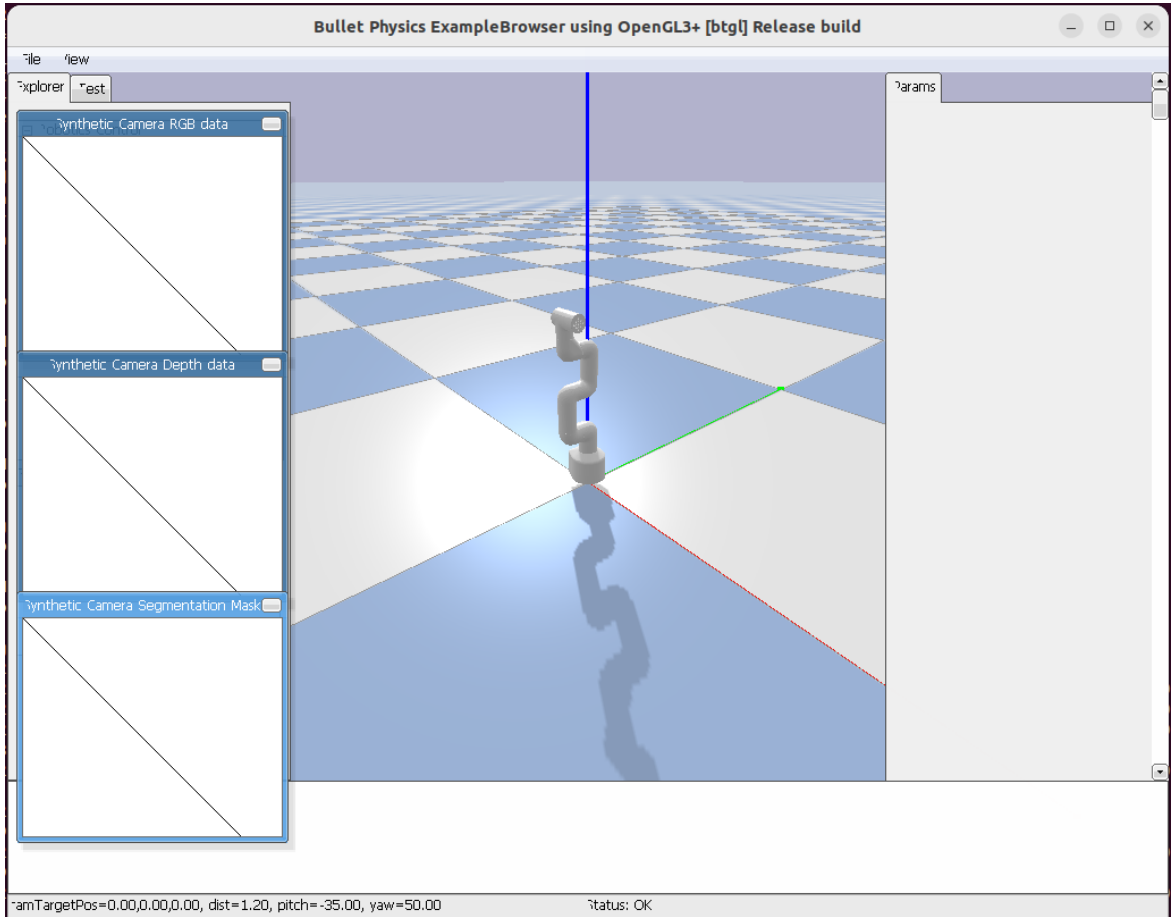
publish_angles(self, angles: numpy.ndarray[typing.Any, numpy.dtype[float]])

```
Takes command angles in degrees and publishes them to ros.

Args:
    angles (npt.NDArray[float]): _description_
```

myCobot—CobotIK Class

- Note the coordinate system
  - x, y, z
- Position units are meters!
- Robot IK commands are in World Coordinate System
- Robot IK commands use Euler angles in degrees as input



myCobot—CobotIK Class

```
28
29    cobot_ik = CobotIK()
30    # Spin in a separate thread
31    thread = threading.Thread(
32        target=rclpy.spin, args=(cobot_ik, ), daemon=True)
33    thread.start()
34
35    demo_time = 60
36    start_time = time.time()
37    loop_rate = 30
38    loop_seconds = 1 / loop_rate
39    speed = 30
40
41    rate = cobot_ik.create_rate(loop_rate)
42
43    zero_msg = get_zero_joints_msg(speed)
44    cobot_ik.cmd_angle_pub.publish(zero_msg)
45
46    time.sleep(5)
47    frame = "gripper"
48
49    cur_angles = cobot_ik.get_real_angles()
50
51    while rclpy.ok() and time.time() - start_time < demo_time:
52
53        x = 0.2
```

myCobot—Starting the Detection App on the Client

- README here, ROS tutorials here are useful
- Control the arm via a script
  - `cd ~/mycobot_client`
  - `colcon build`
  - `source install/setup.bash`
  - `export ROS_DOMAIN_ID=10`
  - `ros2 run mycobot_client_2 run_detection`

myCobot—Starting the Detection Client

- README here, ROS tutorials here are useful
- Control the arm via a script
  - `cd ~/mycobot_client`
  - `colcon build`
  - `source install/setup.bash`
  - `export ROS_DOMAIN_ID=10`
  - `ros2 run mycobot_client_2 run_task_vision`

myCobot—Camera Calculator Class

mycobot_client_2 > mycobot_client_2 > run_detection.py > main

```python
def main(args=None):

    rclpy.init(args=args)

    camera_calculator = CameraCalculator()

    thread = threading.Thread(
        target=rclpy.spin, args=(camera_calculator, ), daemon=True)
    thread.start()

    rate = camera_calculator.create_rate(1)

    while rclpy.ok():
        img = camera_calculator.get_images()
        if img is None:
            camera_calculator.get_logger().error("frame was None")
            time.sleep(0.1)
            continue
        # TODO: add logic to detect cubes, perhaps by HSV color and finding contours, and publish the coordinates via camera_ca
        out_img = cv2.cvtColor(img.color, cv2.COLOR_RGB2HSV)
        found_obj = False
        # setting u, v, to center of screen for example purposes
        u, v = img.color.shape[1]//2, img.color.shape[0]//2
        camera_frame_coordinates, world_frame_coordinates = camera_calculator.get_3d_points_from_pixel_point_on_color(img, u, v
        if found_obj:
            world_coords = np.array((0.2, 0.2, 0))
            camera_calculator.publish_object_found(world_coords.reshape((3,)))
```

Goals

- Recall robotics topics from last time
- Learn to use the myCobot 280 Robotic Arm
- **Write scripts to accomplish manipulation tasks**

myCobot—Your Tasks Part 1 Kinematics

Using your knowledge of inverse kinematics and direct kinematics, we have tasks for you.

1. Pick up and Place Cube—Known Position
    1. Place 1 cube on the grid paper in a known position.
    2. Place the trash can on the grid paper in a known position.
    3. Write a script to pick up the cube and put it into the trash can.
    4. When done call over a supervisor to time your runs.

Tips

- Start slow, understand the coordinate system.
- Start with position only, not orientation. Then use orientation.
- If the arm isn’t reaching a pose, try adjusting orientation.
- Stay close to the e-stop.

myCobot—Your Tasks Part 2

Using your knowledge of computer vision and coordinate frames, we have a task for you.

1. Pick up and Place Cube—Unknown Position
    1. Start the comms node, with the use_realsense argument set to True.
    2. If helpful, use rviz2 to add the camera image by topic and visualize it
    3. Modify run_detection.py to find the cube and publish it's world coordinates.
        - Images will be written to "~/img_plots" by default. So open a file browser and navigate there and open an image to visualize your filters.
        - Echo the ros2 topic to ensure it works "ros2 topic echo /camera/object_found"
        - Cubes aligned with the center line of the robot should have a y coordinate of zero. Cubes are on the table so should have a z value of less than 40mm.
2. Bring up the run_task_vision program and put your pickup code into the CobotIk class inside the pickup_object function. Get the robot to pickup the cube.
3. Get the robot to put it in the trash can.

myCobot—Saving your work

To copy code or images off of the robot, you can connect to the robot's wifi and use a secure copy command. Linux/mac use as is, Windows use from Git bash.

"scp -r er@10.42.0.1:/home/er/img_plots /home/me/img_plots

You may also consider having a teammate maintain a second copy of code on a laptop as you change code on the robot.